

Semaphore

Semaphores in Linux are sleeping locks

Atomic → Protect single variable

Spinlock → Protect short critical section(NO sleeping allowed)

Semaphore → Protect long critical section (Sleeping allowed)

1. Introduction

A **Semaphore** is a synchronization mechanism used to protect shared resources when the critical section may take a long time to execute.

Unlike spinlocks, a task waiting for a semaphore **does not consume CPU cycles**. Instead, it goes to sleep and is awakened when the resource becomes available.

2. Why Do We Need Semaphores?

Spinlocks are suitable only for short critical sections.

Example:

```
spin_lock(&lock);
```

```
/* Critical Section */
```

```
spin_unlock(&lock);
```

A task holding a spinlock:

- Cannot sleep
- Cannot block
- Must release the lock quickly

For long operations such as:

- Hardware I/O
- copy_to_user()
- copy_from_user()
- kmalloc(GFP_KERNEL)
- msleep()

Semaphores are preferred.

3. What is a Semaphore?

A semaphore is a synchronization mechanism that controls access to shared resources using an internal counter.

When a task tries to acquire a busy semaphore:

Task Requests Resource



Resource Busy



Task Sleeps



Added to Wait Queue



Awakened Later

4. How a Semaphore Works

Suppose:

Semaphore Count = 1

Task A acquires semaphore:

Count = 0

Task B attempts to acquire semaphore:

Resource Busy



Task Sleeps



Wait Queue

Task A releases semaphore:

Count = 1



Wake Up Task B

5. Wait Queue

A semaphore maintains a wait queue.

Semaphore



+- Task 1

+- Task 2

+- Task 3

When the semaphore is released:

Wake one waiting task

6. Types of Semaphores

A. Counting Semaphore

Allows multiple tasks to enter.

Initialization:

```
sema_init(&sem, N);
```

Example:

```
sema_init(&sem, 3);
```

Meaning:

Three tasks can access the resource simultaneously.

Count >1

B. Binary Semaphore

Allows only one task at a time.

Initialization:

```
sema_init(&sem, 1);
```

Meaning:

Only one task can enter the critical section.

Count =1

7. Header File

```
#include <linux/semaphore.h>
```

8. Declaring a Semaphore

```
struct semaphore my_sem;
```

9. Initializing a Semaphore

Binary Semaphore

```
sema_init(&my_sem, 1);
```

Counting Semaphore

```
sema_init(&my_sem, 5);
```

10. Acquiring a Semaphore (down –decrement the count)

down()

```
down(&my_sem);
```

If semaphore unavailable:

Task Sleeps

State:

TASK_UNINTERRUPTIBLE

Cannot be interrupted by signals.

down_interruptible()

Preferred in device drivers.

```
if (down_interruptible(&my_sem))  
    return -ERESTARTSYS;
```

State:

TASK_INTERRUPTIBLE

Can be interrupted by signals such as:

Ctrl + C

kill

Returns non-zero if interrupted.

11. Releasing a Semaphore (up- increment the count)

```
up(&my_sem);
```

Operations:

Increase Counter

↓

Wake Waiting Task

12. Example

```
struct semaphore my_sem;
```

```
sema_init(&my_sem, 1);

if (down_interruptible(&my_sem))
    return -ERESTARTSYS;

/* Critical Section */

up(&my_sem);
```

13. Sleeping is Allowed

Valid:

```
down(&my_sem);

msleep(5000);

up(&my_sem);
```

Reason:

Semaphore supports sleeping.

14. Blocking Operations are Allowed

Valid:

```
down(&my_sem);

copy_to_user(...);

up(&my_sem);
```

Valid:

```
down(&my_sem);

copy_from_user(...);

up(&my_sem);
```

Valid:

```
down(&my_sem);

ptr = kmalloc(1024, GFP_KERNEL);

up(&my_sem);
```

15. Process Context Only

Semaphores can only be used in:

Process Context

Examples:

- System Calls
- Driver Read
- Driver Write
- Kernel Threads

16. Cannot Be Used in Interrupt Context

Wrong:

```
irq_handler()
{
    down(&my_sem);
}
```

Reason:

Interrupt Handlers Cannot Sleep

A semaphore may put the task to sleep.

Therefore:

Semaphore NOT allowed in ISR

17. Preemption

Unlike spinlocks:

Spinlock → Preemption Disabled

Semaphore → Preemption Allowed

A task holding a semaphore can be preempted by the scheduler.

18. Semaphore Workflow

down()

↓

Acquire Semaphore

↓
Enter Critical Section
↓
Sleep Allowed
↓
up()

19. Spinlock vs Semaphore

Feature	Spinlock	Semaphore
Waiting Method	Busy Wait	Sleep
CPU Usage	High	Low
Sleeping Allowed	No	Yes
Interrupt Context	Yes	No
Preemption	Disabled	Allowed
Critical Section	Short	Long
Speed	Fast	Slower

20. Advantages

- No CPU wastage.
- Suitable for long-duration operations.
- Supports blocking operations.
- Supports sleeping.
- Efficient resource sharing.

21. Disadvantages

- Context-switch overhead.
- Slower than spinlocks.
- Cannot be used in interrupt handlers.
- More scheduler involvement.

22. Real Usage

Semaphores are commonly used for:

- Device Read Operations
- Device Write Operations
- Shared Buffers
- Resource Pools
- Long Hardware Transactions
- User Space Data Transfers

23. Key Points

1. Semaphore is a sleeping lock.
2. Waiting tasks are placed in a wait queue.
3. Supports long critical sections.
4. `down()` acquires the semaphore.
5. `up()` releases the semaphore.
6. `down_interruptible()` is preferred in drivers.
7. Sleeping is allowed while holding a semaphore.
8. Cannot be used in interrupt context.
9. Preemption remains enabled.
10. Best suited for process context synchronization.

What is a Semaphore?

A semaphore is a synchronization mechanism that allows tasks to sleep while waiting for a shared resource.

Why is a semaphore preferred over a spinlock for long operations?

Because waiting tasks sleep instead of continuously spinning and wasting CPU cycles.

Can a semaphore be used inside an interrupt handler?

No. Interrupt handlers cannot sleep, whereas semaphore acquisition may put a task to sleep.

Difference between `down()` and `down_interruptible()`?

`down()`

→ Uninterruptible Sleep

down_interruptible()
→ Interruptible Sleep

Difference between Binary and Counting Semaphore?

Binary Semaphore

Count = 1

One Task Allowed

→ Mutual Exclusion

→ Protects Critical Section

Counting Semaphore

Count = N

N Tasks Allowed

→ Resource Management

→ Limits number of users

down_trylock() in Linux Semaphore

down_trylock() is similar to spin_trylock().

It tries to acquire the semaphore **without sleeping**.

Function	If Semaphore Busy
down()	Sleep
down_interruptible()	Sleep (interruptible)
down_trylock()	Return immediately

What is the difference between down() and down_trylock()?

Answer:

- down() acquires the semaphore and sleeps if it is unavailable.
- down_trylock() attempts to acquire the semaphore once and immediately returns if it is unavailable.
- down_trylock() never puts the task to sleep.

down_killable() in Linux Semaphore

- down_killable() is another way to acquire a semaphore.
- It is similar to down_interruptible(), but it only wakes up for **fatal signals**.

`down()` → Never interrupt

`down_interruptible()` → Any signal interrupts

`down_killable()` → Fatal signals only

`down_trylock()` → No waiting

What is a fatal signal?

A fatal signal is a signal that causes a process to terminate. Examples include SIGKILL, SIGTERM, SIGQUIT, and SIGABRT. Tasks waiting in TASK_KILLABLE state can be awakened by these signals, which is why `down_killable()` can return early when a fatal signal is received.

1. Semaphores are well suited for locks held for a long time.

- If the semaphore is not available, the waiting task **sleeps** instead of consuming CPU cycles.
- Suitable for long critical sections and blocking operations.

2. Semaphores are not suitable for locks held for a short time.

This is because semaphore operations involve overhead such as:

- Sleeping
- Maintaining the wait queue
- Waking up waiting tasks

Total semaphore overhead can easily exceed the actual lock hold time.

Therefore, for short critical sections, **spinlocks are preferred**.

3. Semaphores cannot be used in interrupt context.

- If the semaphore is unavailable, the task must sleep.
- Interrupt handlers (ISR) cannot sleep.
- Therefore semaphores must be used only in **process context**.

4. Semaphores do not disable kernel preemption.

- A task holding a semaphore can be preempted by the scheduler.
- Unlike spinlocks, preemption remains enabled.

Advantages of Semaphores Over Spinlocks

1. Better CPU Utilization

- No busy waiting.
- Waiting tasks sleep instead of continuously polling.

2. Suitable for Long Critical Sections

- Can hold the lock for a longer duration without wasting CPU cycles.

3. Sleeping Operations are Allowed

Examples:

```
msleep();  
copy_to_user();  
copy_from_user();  
kmalloc(..., GFP_KERNEL);
```

Quick Notes

Semaphore:

- ✓ Long critical sections
- ✓ Sleeping allowed
- ✓ Process context only
- ✓ Better CPU utilization
- ✓ Preemption enabled

Spinlock:

- ✓ Short critical sections
- ✗ Sleeping not allowed
- ✓ Interrupt context allowed
- ✓ Preemption disabled
- ✓ Busy waiting

Why are semaphores preferred for long critical sections?

Answer:

Because waiting tasks sleep instead of busy-waiting, resulting in better CPU utilization. Although semaphores have sleep/wakeup overhead, they are more efficient than spinlocks when the lock is held for a long duration.

```

vboxuser@ubuntu2204:~/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
You are using: gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
CC [M] /home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE/semaphore_demo.o
MODPOST /home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE/Module.symvers
CC [M] /home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE/semaphore_demo.mod.o
LD [M] /home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE/semaphore_demo.ko
BTF [M] /home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE/semaphore_demo.ko
Skipping BTF generation for /home/vboxuser/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE/semaphore_demo.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@ubuntu2204:~/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE$ ls
Makefile      Module.symvers  semaphore_demo.c  semaphore_demo.mod  semaphore_demo.mod.o
modules.order  README.md       semaphore_demo.ko  semaphore_demo.mod.c  semaphore_demo.o
vboxuser@ubuntu2204:~/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE$ sudo insmod semaphore_demo.ko
[sudo] password for vboxuser:
sorry, try again.
[sudo] password for vboxuser:
sorry, try again.
[sudo] password for vboxuser:
sudo: 3 incorrect password attempts
vboxuser@ubuntu2204:~/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE$ sudo insmod semaphore_demo.ko
[sudo] password for vboxuser:
vboxuser@ubuntu2204:~/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE$ sudo dmesg | tail -8
[ 493.377086] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 64 times, consider switching to WQ_UNBOUND
[ 4194.841267] Semaphore Module Loaded
[ 4194.841963] Thread1: Waiting for semaphore
[ 4194.841968] Thread1: Acquired semaphore
[ 4195.863822] Thread2: Waiting for semaphore
[ 4199.964561] Thread1: Releasing semaphore
[ 4199.964605] Thread2: Acquired semaphore
[ 4202.008723] Thread2: Releasing semaphore
vboxuser@ubuntu2204:~/synchronizaaton/SEMAPHORE/BINARY_SEMAPHORE$ S

```

```

vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ nano Makefile
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ nano README.md
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ nano README.md
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ make
make -C /lib/modules/6.8.0-124-generic/build M=/home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-124-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
You are using:          gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
CC [M] /home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE/counting_semaphore_demo.o
MODPOST /home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE/Module.symvers
CC [M] /home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE/counting_semaphore_demo.o
LD [M] /home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE/counting_semaphore_demo.ko
BTF [M] /home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE/counting_semaphore_demo.ko
Skipping BTF generation for /home/vboxuser/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE/counting_semaphore_demo.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-124-generic'
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ ls
counting_semaphore_demo.c  counting_semaphore_demo.mod.c  Makefile  README.md
counting_semaphore_demo.ko  counting_semaphore_demo.mod.o  modules.order
counting_semaphore_demo.mod  counting_semaphore_demo.o  Module.symvers
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ sudo rmmod se
semaphore_demo serio_raw
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ sudo rmmod semaphore_demo
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ sudo insmod counting_semaphore_demo.ko
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ sudo dmesg | tail -15
[ 4525.667885] Thread-2: Waiting for semaphore

```

```
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ sudo dmesg | tail -25
[ 493.377086] workqueue: blk_mq_run_work_fn hogged CPU for >10000us 64 times, consider switching to WQ_UNBOUND
[ 4194.841267] Semaphore Module Loaded
[ 4194.841963] Thread1: Waiting for semaphore
[ 4194.841968] Thread1: Acquired semaphore
[ 4195.863822] Thread2: Waiting for semaphore
[ 4199.964561] Thread1: Releasing semaphore
[ 4199.964605] Thread2: Acquired semaphore
[ 4202.008723] Thread2: Releasing semaphore
[ 4505.751674] Semaphore Module Unloaded
[ 4525.667091] Counting Semaphore Module Loaded
[ 4525.667885] Thread-2: Waiting for semaphore
[ 4525.667890] Thread-2: Acquired semaphore
[ 4525.667947] Thread-3: Waiting for semaphore
[ 4525.667950] Thread-3: Acquired semaphore
[ 4525.669052] Thread-1: Waiting for semaphore
[ 4525.669056] Thread-1: Acquired semaphore
[ 4525.671242] Thread-4: Waiting for semaphore
[ 4525.671937] Thread-5: Waiting for semaphore
[ 4530.714519] Thread-1: Releasing semaphore
[ 4530.714519] Thread-2: Releasing semaphore
[ 4530.715047] Thread-5: Acquired semaphore
[ 4530.715054] Thread-4: Acquired semaphore
[ 4530.715065] Thread-3: Releasing semaphore
[ 4535.833947] Thread-5: Releasing semaphore
[ 4535.834228] Thread-4: Releasing semaphore
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ sudo rmmod counting_semaphore_demo
vboxuser@ubuntu2204:~/synchronizaation/SEMAPHORE/COUNTING_SEMAPHORE$ ss
```